

Vulkan Notes

Pipeline graphique, Shaders & Fixed Functions

Adam HAMOUC

June 2026

1. Pipeline graphique Vulkan

Le pipeline graphique Vulkan est **quasiment immuable** une fois cree. Contrairement a OpenGL qui permettait de modifier shaders et etats a la volee, en Vulkan tout est fige dans un objet **VkPipeline**. Changer un shader, un blend mode ou le fill mode = creer un nouveau pipeline.

Avantage : le driver ne valide plus l'etat a chaque draw call. Pas de surprise au runtime. Meilleures performances.

En pratique : on trie les draw calls par pipeline avant de les soumettre pour minimiser les switches.

Stage	Type	Role	Programmable
Input Assembler	Fixe	Collecte vertices et indices	Non
Vertex Shader	Programmable	Model space → clip space (MVP)	Oui
Tessellation	Programmable	Subdivise la geometrie (Hull → Tess → Domain)	Oui
Geometry Shader	Programmable	Genere/modifie des primitives	Oui
Rasterization	Fixe	Primitives → fragments (pixels potentiels)	Non
Fragment Shader	Programmable	Calcule la couleur finale de chaque fragment	Oui
Color Blending	Fixe	Mixe les fragments superposes	Non

Plusieurs pipelines, plusieurs passes :

- Shader different → pipeline different
- Cible de rendu differente → render pass differente
- Les deux changent → les deux changent

2. Shader Modules et SPIR-V

Un shader module (**VkShaderModule**) est un conteneur Vulkan qui encapsule du bytecode **SPIR-V**.

GLSL (lisible) → **glslc/shaderc** → **SPIR-V** (bytecode) → **VkShaderModule** → **VkPipeline**

Avantages de SPIR-V sur GLSL compile au runtime (OpenGL) :

- Compile offline, comportement identique sur tous les drivers/GPU
- Bas niveau : pas de differences d'interpretation entre vendeurs
- Validation et optimisation possibles avant le deploiement

```
// Chargement et creation du shader module
auto spirvCode = readFile("pbr.frag.spv");
VkShaderModuleCreateInfo createInfo{};
createInfo.sType      = VK_STRUCTURE_TYPE_SHADER_MODULE_CREATE_INFO;
createInfo.codeSize    = spirvCode.size();
createInfo.pCode       = reinterpret_cast<const uint32_t*>(spirvCode.data());
VkShaderModule fragModule;
vkCreateShaderModule(device, &createInfo, nullptr, &fragModule);
```

```
// Apres creation du pipeline, le module peut etre detruit immediatement
vkCreateGraphicsPipelines(device, pipelineCache, 1, &pipelineInfo, nullptr, &pipeline);
vkDestroyShaderModule(device, fragModule, nullptr); // plus necessaire
```

Points d'entree multiples : un seul fichier SPIR-V peut contenir plusieurs fonctions "main" avec des noms differents. On specifie lequel utiliser via `pName` dans `VkPipelineShaderStageCreateInfo`. Utile pour partager du code utilitaire entre variantes de shaders. Rare en pratique.

3. Fixed Functions — Structs a configurer

En OpenGL, les etapes a fonction fixee avaient des valeurs par default. En Vulkan **aucune valeur par default** — tout doit etre configure explicitement via des structs avant la creation du pipeline.

```
// Vertex input : format des donnees par vertex (position, UV, normale...)
// Supporte aussi l'instancing : VK_VERTEX_INPUT_RATE_INSTANCE
VkPipelineVertexInputStateCreateInfo vertexInput{};

// Input assembly : type de primitive (triangles, lignes, points)
VkPipelineInputAssemblyStateCreateInfo inputAssembly{};
inputAssembly.topology = VK_PRIMITIVE_TOPOLOGY_TRIANGLE_LIST;

// Viewport et scissor : region de rendu dans le framebuffer
VkPipelineViewportStateCreateInfo viewportState{};

// Rasterization : fill mode (solid/wireframe), face culling, depth bias
VkPipelineRasterizationStateCreateInfo rasterizer{};
rasterizer.cullMode = VK_CULL_MODE_BACK_BIT;
rasterizer.depthBias = 1.75f; // utile pour shadow maps

// Multisampling (MSAA) : plusieurs samples par pixel pour antialiaser les bords
// Fragment shader invoque 1x par pixel, test de couverture fait N fois
// Bien plus efficace que le supersampling (SSAA)
VkPipelineMultisampleStateCreateInfo multisampling{};

// Depth/Stencil : depth test, depth write, comparaison
VkPipelineDepthStencilStateCreateInfo depthStencil{};
depthStencil.depthTestEnable = VK_TRUE;
depthStencil.depthWriteEnable = VK_TRUE;
depthStencil.depthCompareOp = VK_COMPARE_OP_LESS;

// Color blending : mixage des fragments superposes (transparence)
VkPipelineColorBlendStateCreateInfo colorBlending{};
```

4. Pipeline Layout

Le pipeline layout decrit les **ressources externes** auxquelles le pipeline peut acceder — tout ce que le shader lit en dehors des vertex buffers.

Descriptor set layouts : structure des UBO, textures, buffers lies par sets et bindings.

Push constants : petites donnees envoyees directement dans le command buffer (model matrix, material ID...). Ultra rapide, limite en taille (128 bytes).

Important : le layout declare la structure (le contrat). Les donnees reelles arrivent via les descriptor sets binded au moment du draw call.

```
// Cote shader
layout(set = 0, binding = 0) uniform CameraUBO { mat4 view; mat4 proj; } camera;
layout(set = 1, binding = 0) uniform sampler2D albedoMap;
layout(push_constant) uniform Push { mat4 modelMatrix; } push;
```

```
// Cote C++ : pipeline layout = contrat entre le pipeline et ses ressources
VkPipelineLayoutCreateInfo layoutInfo{};
layoutInfo.setLayoutCount      = descriptorSetLayouts.size();
layoutInfo.pSetLayouts        = descriptorSetLayouts.data();
layoutInfo.pushConstantRangeCount = pushConstantRanges.size();
layoutInfo.pPushConstantRanges = pushConstantRanges.data();
vkCreatePipelineLayout(device, &layoutInfo, nullptr, &pipelineLayout);
```

5. Render Pass

La render pass declare la **structure** d'une passe de rendu : quelles images sont utilisees (attachments), comment elles sont traitees en entree (**loadOp**) et en sortie (**storeOp**).

LOAD_OP_CLEAR	Efface l'image avant de dessiner (nouvelle frame)
LOAD_OP_LOAD	Conserve le contenu existant (accumulation)
STORE_OP_STORE	Garde le resultat pour la passe suivante
STORE_OP_DONT_CARE	Jette le resultat (depth buffer apres le rendu)

La render pass ne contient pas les draw calls ni les shaders — c'est le plan. Le framebuffer fournit les vraies images, le pipeline les shaders, le command buffer les draw calls.

Ordre de destruction : Pipeline → Pipeline Layout → Render Pass → Framebuffers → ImageViews → SwapChain → Surface → Device → Instance.